In [ ]:
```python
import marimo as mo
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn

from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms
```

SNN на основе модели Ходжкина — Хаксли

Классификация рукописных цифр MNIST с помощью биофизической модели нейрона.

Архитектура

- Вход: 784 пикселя.
- Скрытый слой: 128 нейронов HH.
- Выходной слой: 10 нейронов HH.
- Модельное время: 5 мс.
- Шаг интегрирования: 0,05 мс.
- Обучение: PyTorch + суррогатный градиент.
- Классификация: по числу выходных спайков.

Динамика мембраны

$$C_m \frac{dV}{dt} = I_{\text{ext}} - I_{\text{Na}} - I_{\text{K}} - I_{\text{L}}$$

$$I_{\text{Na}} = \bar{g}_{\text{Na}} m^3 h (V - E_{\text{Na}})$$

$$I_{\text{K}} = \bar{g}_{\text{K}} n^4 (V - E_{\text{K}})$$

$$I_{\text{L}} = g_{\text{L}} (V - E_{\text{L}})$$

Переменные m, h, n описывают состояние ионных каналов. При достижении порога 0 мВ регистрируется спайк. Сам мембранный потенциал при этом не сбрасывается.

In [ ]:
```python
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Устройство:", device)
```

```
if device.type == "cuda":
    print("GPU:", torch.cuda.get_device_name(device))
```

```
Устройство: cuda
GPU: NVIDIA GeForce RTX 3070 Ti Laptop GPU
```

In [ ]:

```
transform = transforms.ToTensor()

train_full = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform,
)

test_full = datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform,
)

# Размеры можно увеличить для более полного эксперимента.
TRAIN_SIZE = 6000
TEST_SIZE = 1000

train_set = Subset(
    train_full,
    range(min(TRAIN_SIZE, len(train_full))),
)

test_set = Subset(
    test_full,
    range(min(TEST_SIZE, len(test_full))),
)

train_loader = DataLoader(
    train_set,
    batch_size=64,
    shuffle=True,
    num_workers=0,
)

test_loader = DataLoader(
    test_set,
    batch_size=128,
    shuffle=False,
    num_workers=0,
)

print("Обучающая выборка:", len(train_set))
print("Тестовая выборка:", len(test_set))
```

```
Обучающая выборка: 6000
Тестовая выборка: 1000
```

In [ ]:
```
class SpikeFunction(torch.autograd.Function):
    """
    Прямой проход:
        spike = 1, если V > 0 мВ;
        spike = 0, иначе.

    Обратный проход:
        гладкая аппроксимация производной пороговой функции.
    """

    @staticmethod
    def forward(ctx, voltage, slope):
        ctx.save_for_backward(voltage)
        ctx.slope = slope

        return (voltage > 0.0).to(voltage.dtype)

    @staticmethod
    def backward(ctx, grad_output):
        (voltage,) = ctx.saved_tensors
        slope = ctx.slope

        grad_voltage = (
            slope
            / (1.0 + slope * voltage.abs()).square()
        )

        return grad_output * grad_voltage, None

def make_spike(voltage, slope=5.0):
    return SpikeFunction.apply(voltage, slope)
```

In [ ]:
```
class HodgkinHuxleyLayer(nn.Module):
    """
    Параллельная модель HH для произвольного количества
    независимых нейронов.

    Вход:
        current [batch, neurons] — внешний ток, мкА/см².

    Состояние каждого нейрона:
        V — мембранный потенциал, мВ;
        m — активация натриевых каналов;
        h — инактивация натриевых каналов;
        n — активация калиевых каналов.

    Интегрирование:
        явный метод Эйлера.
    """

    def __init__(
        self,
        neurons,
        dt=0.05,
        num_steps=100,
```

```
        cm=1.0,
        g_na=120.0,
        g_k=36.0,
        g_l=0.3,
        e_na=50.0,
        e_k=-77.0,
        e_l=-54.387,
    ):
        super().__init__()

        self.neurons = neurons
        self.dt = dt
        self.num_steps = num_steps

        self.cm = cm
        self.g_na = g_na
        self.g_k = g_k
        self.g_l = g_l

        self.e_na = e_na
        self.e_k = e_k
        self.e_l = e_l

    @staticmethod
    def _vtrap(x, scale, limit):
        """
        Устойчивая форма:
            x / (1 - exp(-x / scale))

        В точке x = 0 используется аналитический предел.
        """
        small = x.abs() < limit

        x_safe = torch.where(
            small,
            torch.ones_like(x),
            x,
        )

        regular = (
            x_safe
            / (-torch.expm1(-x_safe / scale))
        )

        # Разложение в ряд Тейлора около нуля.
        series = (
            scale
            + x / 2.0
            + x.square() / (12.0 * scale)
        )

        return torch.where(small, series, regular)

    @classmethod
    def alpha_m(cls, v):
        return 0.1 * cls._vtrap(
```

```
            v + 40.0, 10.0, 1e-4
        )

    @staticmethod
    def beta_m(v):
        return 4.0 * torch.exp(
            (-(v + 65.0) / 18.0).clamp(-50, 50)
        )

    @staticmethod
    def alpha_h(v):
        return 0.07 * torch.exp(
            (-(v + 65.0) / 20.0).clamp(-50, 50)
        )

    @staticmethod
    def beta_h(v):
        return 1.0 / (
            1.0 + torch.exp(
                (-(v + 35.0) / 10.0).clamp(-50, 50)
            )
        )

    @classmethod
    def alpha_n(cls, v):
        return 0.01 * cls._vtrap(
            v + 55.0, 10.0, 1e-4
        )

    @staticmethod
    def beta_n(v):
        return 0.125 * torch.exp(
            (-(v + 65.0) / 80.0).clamp(-50, 50)
        )

    def steady_state(self, v):
        """
        Равновесные значения ворот при заданном V.
        """
        am = self.alpha_m(v)
        bm = self.beta_m(v)

        ah = self.alpha_h(v)
        bh = self.beta_h(v)

        an = self.alpha_n(v)
        bn = self.beta_n(v)

        m = am / (am + bm)
        h = ah / (ah + bh)
        n = an / (an + bn)

        return m, h, n

    def forward(
        self,
```

```
        current,
        state=None,
        return_voltage=False,
    ):
        """
        current:
            [batch, neurons]

        state:
            необязательное начальное состояние (V, m, h, n).

        Если state не задан, каждый нейрон начинается
        из состояния покоя при V = -65 мВ.
        """

        batch_size = current.shape[0]

        if state is None:
            v = current.new_full(
                (batch_size, self.neurons),
                -65.0,
            )

            m, h, n = self.steady_state(v)

        else:
            v, m, h, n = state

        spikes_record = []
        voltage_record = []

        for _ in range(self.num_steps):
            # Ионные токи, мкА/см².
            i_na = (
                self.g_na
                * m.pow(3)
                * h
                * (v - self.e_na)
            )

            i_k = (
                self.g_k
                * n.pow(4)
                * (v - self.e_k)
            )

            i_l = self.g_l * (v - self.e_l)

            # Уравнение мембранного потенциала.
            dv = (
                current - i_na - i_k - i_l
            ) / self.cm

            # Скорости переходов ворот.
            am = self.alpha_m(v)
            bm = self.beta_m(v)
```

```
            ah = self.alpha_h(v)
            bh = self.beta_h(v)

            an = self.alpha_n(v)
            bn = self.beta_n(v)

            dm = am * (1.0 - m) - bm * m
            dh = ah * (1.0 - h) - bh * h
            dn = an * (1.0 - n) - bn * n

            dt = self.dt

            # Интегрирование методом Эйлера.
            v = v + dt * dv
            m = (m + dt * dm).clamp(0.0, 1.0)
            h = (h + dt * dh).clamp(0.0, 1.0)
            n = (n + dt * dn).clamp(0.0, 1.0)

            # Ограничение служит численной защитой,
            # а не физиологическим сбросом потенциала.
            v = v.clamp(-100.0, 60.0)

            spike = make_spike(v, slope=5.0)

            spikes_record.append(spike)

            if return_voltage:
                voltage_record.append(v)

        spikes = torch.stack(spikes_record, dim=0)

        final_state = (v, m, h, n)

        if return_voltage:
            voltages = torch.stack(
                voltage_record,
                dim=0,
            )

            return spikes, voltages, final_state

        return spikes, final_state

class HodgkinHuxleyNet(nn.Module):
    """
    Сеть:
        784 -> линейное преобразование -> HH(128)
            -> линейное преобразование -> HH(10).

    Каждый слой HH интегрируется во времени.
    """

    def __init__(
        self,
        input_size=784,
```

```
        hidden=128,
        classes=10,
        dt=0.05,
        num_steps=100,
    ):
        super().__init__()

        self.num_steps = num_steps
        self.dt = dt

        self.fc1 = nn.Linear(input_size, hidden)
        self.fc2 = nn.Linear(hidden, classes)

        self.hh1 = HodgkinHuxleyLayer(
            neurons=hidden,
            dt=dt,
            num_steps=num_steps,
        )

        self.hh2 = HodgkinHuxleyLayer(
            neurons=classes,
            dt=dt,
            num_steps=num_steps,
        )

        # Масштабы переводят линейные сигналы в диапазон
        # внешних токов HH, мкА/см².
        self.input_scale = 20.0
        self.output_scale = 20.0

        # Инициализация весов под выбранный масштаб токов.
        nn.init.normal_(self.fc1.weight, mean=0.0, std=0.05)
        nn.init.zeros_(self.fc1.bias)

        nn.init.normal_(self.fc2.weight, mean=0.0, std=0.10)
        nn.init.zeros_(self.fc2.bias)

    def forward(
        self,
        images,
        return_voltage=False,
    ):
        x = images.reshape(images.shape[0], -1)

        # Ток скрытого слоя постоянен в течение
        # одного моделирования изображения.
        current1 = self.input_scale * self.fc1(x)

        if return_voltage:
            spikes1, voltage1, _ = self.hh1(
                current1,
                return_voltage=True,
            )
        else:
            spikes1, _ = self.hh1(current1)
```

```
        output_spikes = []
        output_voltages = []

        # Выходной слой получает новый ток на каждом
        # временном шаге от спайков скрытого слоя.
        v2 = None
        state2 = None

        for step in range(self.num_steps):
            hidden_spikes = spikes1[step]

            current2 = (
                self.output_scale
                * self.fc2(hidden_spikes)
            )

            if state2 is None:
                if return_voltage:
                    spikes2, voltage2, state2 = self.hh2(
                        current2,
                        return_voltage=True,
                    )
                else:
                    spikes2, state2 = self.hh2(current2)

                # Первый вызов HH возвращает целую
                # последовательность, что для сети
                # здесь не нужно. Используем его
                # начальное состояние в отдельном
                # одношаговом интеграторе ниже.
                #
                # Чтобы не моделировать весь интервал
                # повторно, используем отдельный метод
                # одного временного шага.
                state2 = None
                break

        # Совместное пошаговое моделирование слоёв
        # реализуется через метод ниже.
        return self._forward_stepwise(
            x,
            return_voltage=return_voltage,
        )

    def _forward_stepwise(
        self,
        x,
        return_voltage=False,
    ):
        batch_size = x.shape[0]

        v1 = x.new_full(
            (batch_size, self.hh1.neurons),
            -65.0,
        )
        m1, h1, n1 = self.hh1.steady_state(v1)
```

```
        v2 = x.new_full(
            (batch_size, self.hh2.neurons),
            -65.0,
        )
        m2, h2, n2 = self.hh2.steady_state(v2)

        current1 = self.input_scale * self.fc1(x)

        spikes_record = []
        voltage_record = []

        for _ in range(self.num_steps):
            # ----------------------------
            # Скрытый слой HH
            # ----------------------------
            i_na1 = (
                self.hh1.g_na
                * m1.pow(3)
                * h1
                * (v1 - self.hh1.e_na)
            )

            i_k1 = (
                self.hh1.g_k
                * n1.pow(4)
                * (v1 - self.hh1.e_k)
            )

            i_l1 = self.hh1.g_l * (v1 - self.hh1.e_l)

            dv1 = (
                current1 - i_na1 - i_k1 - i_l1
            ) / self.hh1.cm

            am1 = self.hh1.alpha_m(v1)
            bm1 = self.hh1.beta_m(v1)

            ah1 = self.hh1.alpha_h(v1)
            bh1 = self.hh1.beta_h(v1)

            an1 = self.hh1.alpha_n(v1)
            bn1 = self.hh1.beta_n(v1)

            dt = self.dt

            v1 = (v1 + dt * dv1).clamp(-100.0, 60.0)
            m1 = (m1 + dt * (
                am1 * (1.0 - m1) - bm1 * m1
            )).clamp(0.0, 1.0)

            h1 = (h1 + dt * (
                ah1 * (1.0 - h1) - bh1 * h1
            )).clamp(0.0, 1.0)

            n1 = (n1 + dt * (
```

```
            an1 * (1.0 - n1) - bn1 * n1
        )).clamp(0.0, 1.0)

        spike1 = make_spike(v1, slope=5.0)

        # ----------------------------
        # Выходной слой НН
        # ----------------------------
        current2 = (
            self.output_scale * self.fc2(spike1)
        )

        i_na2 = (
            self.hh2.g_na
            * m2.pow(3)
            * h2
            * (v2 - self.hh2.e_na)
        )

        i_k2 = (
            self.hh2.g_k
            * n2.pow(4)
            * (v2 - self.hh2.e_k)
        )

        i_l2 = self.hh2.g_l * (v2 - self.hh2.e_l)

        dv2 = (
            current2 - i_na2 - i_k2 - i_l2
        ) / self.hh2.cm

        am2 = self.hh2.alpha_m(v2)
        bm2 = self.hh2.beta_m(v2)

        ah2 = self.hh2.alpha_h(v2)
        bh2 = self.hh2.beta_h(v2)

        an2 = self.hh2.alpha_n(v2)
        bn2 = self.hh2.beta_n(v2)

        v2 = (v2 + dt * dv2).clamp(-100.0, 60.0)

        m2 = (m2 + dt * (
            am2 * (1.0 - m2) - bm2 * m2
        )).clamp(0.0, 1.0)

        h2 = (h2 + dt * (
            ah2 * (1.0 - h2) - bh2 * h2
        )).clamp(0.0, 1.0)

        n2 = (n2 + dt * (
            an2 * (1.0 - n2) - bn2 * n2
        )).clamp(0.0, 1.0)

        spike2 = make_spike(v2, slope=5.0)
```

```
            spikes_record.append(spike2)

            if return_voltage:
                voltage_record.append(v2)

        spikes = torch.stack(spikes_record, dim=0)

        if return_voltage:
            voltages = torch.stack(
                voltage_record,
                dim=0,
            )

            return spikes, voltages

        return spikes
```

In [ ]:

```
model = HodgkinHuxleyNet(
    input_size=784,
    hidden=128,
    classes=10,
    dt=0.05,
    num_steps=100,
).to(device)

total_params = sum(
    p.numel()
    for p in model.parameters()
    if p.requires_grad
)

print(model)
print("Обучаемых параметров:", total_params)
print("Шаг интегрирования:", model.dt, "мс")
print(
    "Время моделирования:",
    model.dt * model.num_steps,
    "мс",
)
```

```
HodgkinHuxleyNet(
  (fc1): Linear(in_features=784, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
  (hh1): HodgkinHuxleyLayer()
  (hh2): HodgkinHuxleyLayer()
)
Обучаемых параметров: 101770
Шаг интегрирования: 0.05 мс
Время моделирования: 5.0 мс
```

In [ ]:
```
criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.0003,
)
```

In [ ]:
```
def train_one_epoch():
    model.train()

    total_loss = 0.0
    total_correct = 0
    total_seen = 0

    current_device = next(
        model.parameters()
    ).device

    for images, labels in train_loader:
        images = images.to(current_device)
        labels = labels.to(current_device)

        optimizer.zero_grad(set_to_none=True)

        # [time, batch, classes]
        spikes = model(images)

        # Суммарная активность выходных нейронов.
        spike_counts = spikes.sum(dim=0)

        loss = criterion(
            spike_counts,
            labels,
        )

        loss.backward()

        # Дополнительная защита от слишком больших градиентов.
        torch.nn.utils.clip_grad_norm_(
            model.parameters(),
            max_norm=5.0,
        )

        optimizer.step()

        batch_size = labels.size(0)

        total_loss += loss.item() * batch_size

        predictions = spike_counts.argmax(dim=1)

        total_correct += (
            predictions == labels
        ).sum().item()
```

```
        total_seen += batch_size

    return (
        total_loss / max(total_seen, 1),
        total_correct / max(total_seen, 1),
    )
```

In [ ]:
```
@torch.no_grad()
def evaluate():
    model.eval()

    current_device = next(
        model.parameters()
    ).device

    total_correct = 0
    total_seen = 0

    for images, labels in test_loader:
        images = images.to(current_device)
        labels = labels.to(current_device)

        spikes = model(images)

        spike_counts = spikes.sum(dim=0)

        predictions = spike_counts.argmax(dim=1)

        total_correct += (
            predictions == labels
        ).sum().item()

        total_seen += labels.size(0)

    return total_correct / max(total_seen, 1)
```

In [ ]:
```
epochs_slider = mo.ui.slider(
    start=1,
    stop=5,
    value=2,
    step=1,
    label="Количество эпох",
)

training_form = mo.ui.form(
    mo.md("""
    ### Обучение сети

    Выбери число эпох: {epochs}
    """).batch(epochs=epochs_slider),
    submit_button_label="Запустить обучение",
)

training_form
```

Обучение сети

Выбери число эпох: Количество эпох

Запустить обучение

In [ ]:
```
if training_form.value is None:
    mo.md(
        "Выбери число эпох и отправь форму, "
        "чтобы запустить обучение."
    )

else:
    num_epochs = int(
        training_form.value["epochs"]
    )

    results = []

    for epoch in range(1, num_epochs + 1):
        loss_value, train_acc = train_one_epoch()
        test_acc = evaluate()

        results.append({
            "Эпоха": epoch,
            "Loss": round(loss_value, 4),
            "Обучение (%)": round(train_acc * 100, 2),
            "Тест (%)": round(test_acc * 100, 2),
        })

        print(
            f"Эпоха {epoch}/{num_epochs} | "
            f"Loss={loss_value:.4f} | "
            f"Train={train_acc * 100:.2f}% | "
            f"Test={test_acc * 100:.2f}%"
        )

    mo.ui.table(results)
```

```
Эпоха 1/2 | Loss=12.3040 | Train=11.52% | Test=10.70%
Эпоха 2/2 | Loss=3.6896 | Train=10.60% | Test=8.50%
```

In [ ]:
```
show_examples = mo.ui.run_button(
    label="Показать примеры распознавания"
)

show_examples
```

Показать примеры распознавания

In [ ]:
```
if show_examples.value:
    model.eval()
```

```
current_device = next(
    model.parameters()
).device

images, labels = next(iter(test_loader))

images = images[:12].to(current_device)
labels = labels[:12].to(current_device)

with torch.no_grad():
    spikes, voltages = model(
        images,
        return_voltage=True,
    )

    spike_counts = spikes.sum(dim=0)

    predictions = spike_counts.argmax(dim=1)

images_cpu = images.cpu()
labels_cpu = labels.cpu()
predictions_cpu = predictions.cpu()
counts_cpu = spike_counts.cpu()
voltages_cpu = voltages.cpu()

# ------------------------------------------------------
# 14.1. Изображения и результаты
# ------------------------------------------------------

fig, axes = plt.subplots(
    3,
    4,
    figsize=(10, 8),
)

for idx, ax in enumerate(axes.ravel()):
    ax.imshow(
        images_cpu[idx, 0].numpy(),
        cmap="gray",
    )

    true_digit = int(labels_cpu[idx])
    predicted_digit = int(predictions_cpu[idx])

    correct = true_digit == predicted_digit

    ax.set_title(
        f"Сеть: {predicted_digit}\n"
        f"Ответ: {true_digit}\n"
        f"{'Верно' if correct else 'Ошибка'}",
        color="green" if correct else "red",
        fontsize=9,
    )

    ax.axis("off")
```

```
fig.tight_layout()
plt.show()
plt.close(fig)

# ------------------------------------------------------
# 14.2. Количество выходных спайков
# ------------------------------------------------------

fig_counts, ax_counts = plt.subplots(
    figsize=(8, 3),
)

ax_counts.bar(
    range(10),
    counts_cpu[0].numpy(),
)

ax_counts.set_xticks(range(10))
ax_counts.set_xlabel("Цифра")
ax_counts.set_ylabel("Число спайков")
ax_counts.set_title(
    f"Выходная активность для изображения: "
    f"{int(predictions_cpu[0])}"
)

ax_counts.grid(axis="y", alpha=0.3)
fig_counts.tight_layout()
plt.show()
plt.close(fig_counts)

# ------------------------------------------------------
# 14.3. Мембранный потенциал выходных нейронов
# ------------------------------------------------------

# Показываем потенциал нейрона-победителя
# для первого изображения.
first_prediction = int(predictions_cpu[0])

time_axis = (
    torch.arange(voltages_cpu.shape[0]).numpy()
    * model.dt
)

first_voltage = voltages_cpu[
    :,
    0,
    first_prediction,
].numpy()

fig_voltage, ax_voltage = plt.subplots(
    figsize=(9, 4),
)

ax_voltage.plot(
    time_axis,
    first_voltage,
```

```
    lw=1.2,
)

ax_voltage.axhline(
    0.0,
    linestyle="--",
    alpha=0.7,
    label="Порог регистрации спайка",
)

ax_voltage.set_xlabel("Время (мс)")
ax_voltage.set_ylabel("V (мВ)")
ax_voltage.set_title(
    f"Потенциал выходного нейрона {first_prediction}"
)

ax_voltage.grid(True, alpha=0.3)
ax_voltage.legend()
fig_voltage.tight_layout()
plt.show()
plt.close(fig_voltage)

# ------------------------------------------------------
# 14.4. Таблица предсказаний
# ------------------------------------------------------

result_rows = []

for idx in range(len(labels_cpu)):
    true_digit = int(labels_cpu[idx])
    predicted_digit = int(predictions_cpu[idx])

    result_rows.append({
        "№": idx + 1,
        "Ответ": true_digit,
        "Предсказание": predicted_digit,
        "Спайки победителя": int(
            counts_cpu[idx, predicted_digit]
        ),
        "Верно": true_digit == predicted_digit,
    })

mo.ui.table(result_rows)

correct_count = (
    predictions_cpu == labels_cpu
).sum().item()

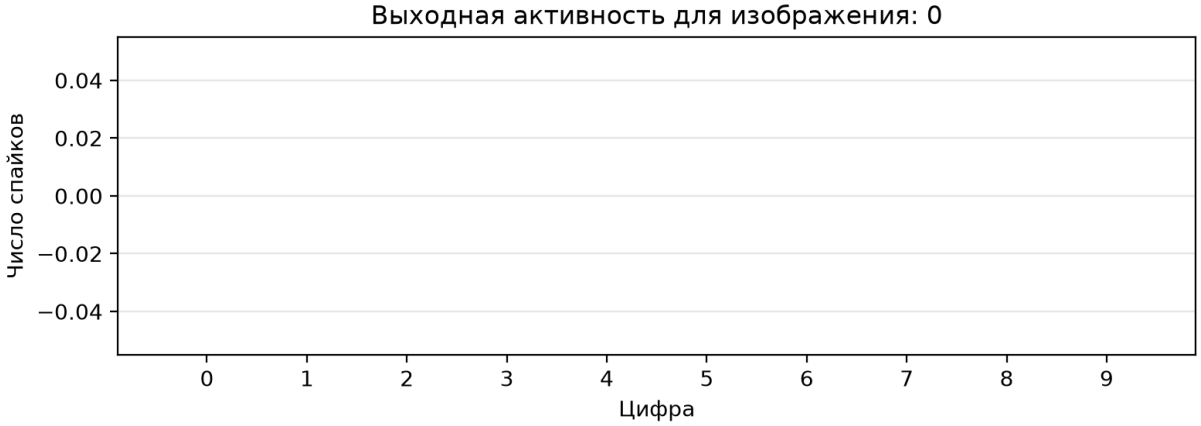
mo.md(
    f"""
    ### Результат

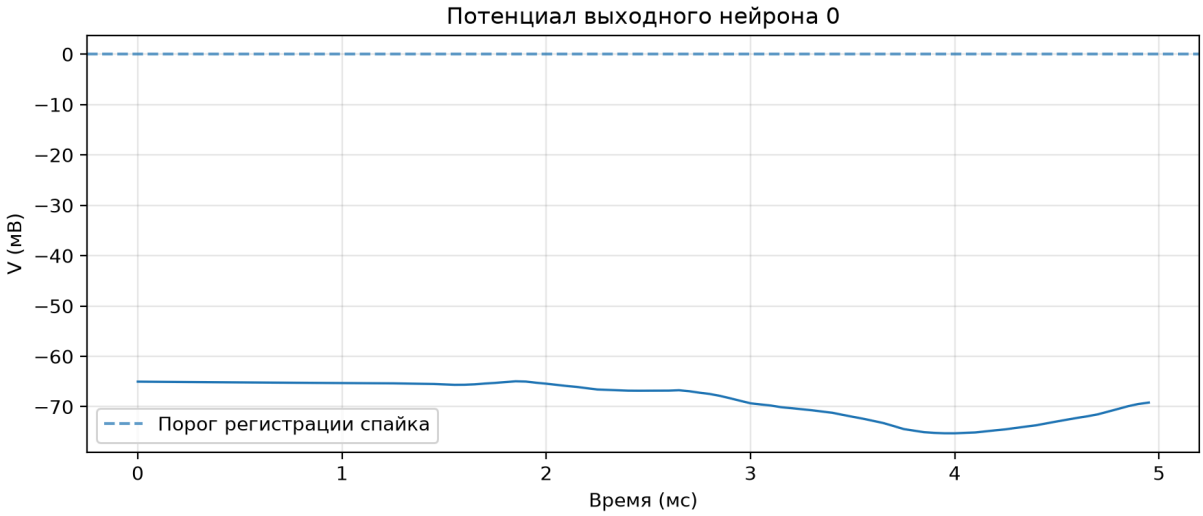
    - Правильно распознано: **{correct_count} из {len(labels_cpu)}**
    - Точность на показанных изображениях:
      **{100 * correct_count / len(labels_cpu):.2f}%**
```

```
        Это результат только на показанных изображениях.
        """
    )

else:
    mo.md(
        "Нажми кнопку, чтобы посмотреть предсказания "
        "и мембранный потенциал выходного нейрона."
    )
```






Потенциал выходного нейрона 0
0
−10
−20
−30
−40
−50
−60
−70
V (мВ)
Порог регистрации спайка
0
1
2
3
4
5
Время (мс)